

Siegfried Lederer game — Informe final desafio 2

Informática II — 2025-2

Cristian bravo y José Manuel Giraldo

1. Introducción

El proyecto “**Siegfried Lederer Game**” es un videojuego 2D desarrollado en C++ usando el framework **Qt** (Qt Widgets + Qt Multimedia) como trabajo final del curso de informática 2. El juego está inspirado en el escape real de Siegfried Lederer del campo de concentración de Auschwitz y se divide en **tres niveles**, cada uno con mecánicas, vistas y físicas distintas: un laberinto cenital, una persecución en carretera y una lluvia de bombas en una trinchera.

El proyecto está principalmente dirigido a estudiantes, profesores y jugadores casuales que disfrutan juegos retro sencillos, con un nivel de dificultad moderado y un contexto histórico que genera curiosidad.

2. Planteamiento del problema

Se buscaba diseñar un videojuego corto que permitiera:

- **Practicar POO, memoria dinámica y GUI** en C++/Qt, según los lineamientos del curso.
- Cumplir con la consigna de tener una temática histórica relevante y al menos tres niveles con físicas diferentes, un agente inteligente y un sistema de dificultad.

La necesidad concreta es disponer de un ejemplo completo donde se integren lógica de juego, IA básica, físicas simples, manejo de recursos gráficos/sonoros y una interfaz amigable, todo en un único proyecto coherente.

3. Definición general y objetivos

Alcance funcional

- Tres niveles jugables, seleccionables desde un menú principal.
- Sistema de vidas, llaves y tiempo.

- Enemigos con **comportamiento autónomo** (guardias del laberinto).
- Físicas:
 - Movimiento rectilíneo uniforme (jugador).
 - Movimiento circular (trampas).
 - Movimiento parabólico (proyectiles del tanque).
 - Caída libre con aceleración (bombas).

Objetivos específicos

- Implementar un videojuego funcional con Qt usando POO y memoria dinámica.
- Diseñar al menos un agente inteligente con percepción, razonamiento, acción y aprendizaje básico (guardias del nivel 1).
- Incorporar al menos tres modelos físicos propios.
- Desarrollar una interfaz gráfica completa (menú, pantallas de victoria y derrota).

4. Especificación de requerimientos

4.1 Requisitos funcionales (resumen)

Nivel 1 – Laberinto cenital

- Mover al prisionero en cuatro direcciones dentro de una grilla de celdas.
- Detectar colisiones con paredes usando un MazeGrid.
- Guardias que patrullan rutas predefinidas y, si el jugador entra en su campo de visión, cambian a modo persecución.
- Campo de visión limitado en distancia y bloqueado por paredes.
- Trampas móviles que restan vida al jugador al colisionar.
- Sistema de llaves (2 azules, 2 blancas) obligatorio para abrir la salida.
- Sistema de vidas compartido con guardias y trampas.

Nivel 2 – Carretera con tanque (vista lateral)

- Jugador que corre automáticamente y puede cambiar entre **tres carriles** (arriba, medio, abajo).
- Obstáculos de alambre de púas ubicados en distintos carriles.
- Tanque en el fondo que dispara proyectiles con trayectoria **parabólica** hacia carriles aleatorios.
- Marcador visual (“mira”) que anuncia el punto de impacto.
- Colisiones que restan vidas y reinician la posición del jugador si aún quedan vidas.

Nivel 3 – Trinchera con bombas

- Jugador que se mueve horizontalmente dentro de un rango limitado.
- Bombas que caen en caída libre con aceleración, desde posiciones X aleatorias.
- Dificultad creciente reduciendo el intervalo de aparición de bombas.
- Barra de tiempo y contador en segundos; objetivo: sobrevivir hasta que el tiempo llegue a cero.

4.2 Requisitos no funcionales

- El juego debe ejecutarse de forma fluida (≈60 FPS) en equipos de nivel medio.
- Interfaz visual coherente con estética ya sea de pixel art / 2D retro entre otras que fácilmente podrían haber sido implementadas.
- Controles simples wasd o las flechas.
- Efectos de sonido y música de fondo al menos en un nivel.
- Código organizado en clases, con uso de memoria dinámica y contenedores (QVector, QList, etc.).

4.3 Restricciones y dependencias

- Uso obligatorio de **Qt** para GUI y multimedia.
- Lenguaje: C++.

- No se permiten niveles plataformeros; las vistas deben ser cenitales, laterales, etc..
- Sprites y recursos adaptados a las dimensiones de las escenas.
- Dependencia de módulos: QtCore, QtGui, QtWidgets, QtMultimedia.

4.4 Usuarios destinatarios

- Docente y estudiantes del curso, para fines de evaluación y aprendizaje.
 - Cualquier jugador casual que disfrute un juego corto tipo arcade.
-

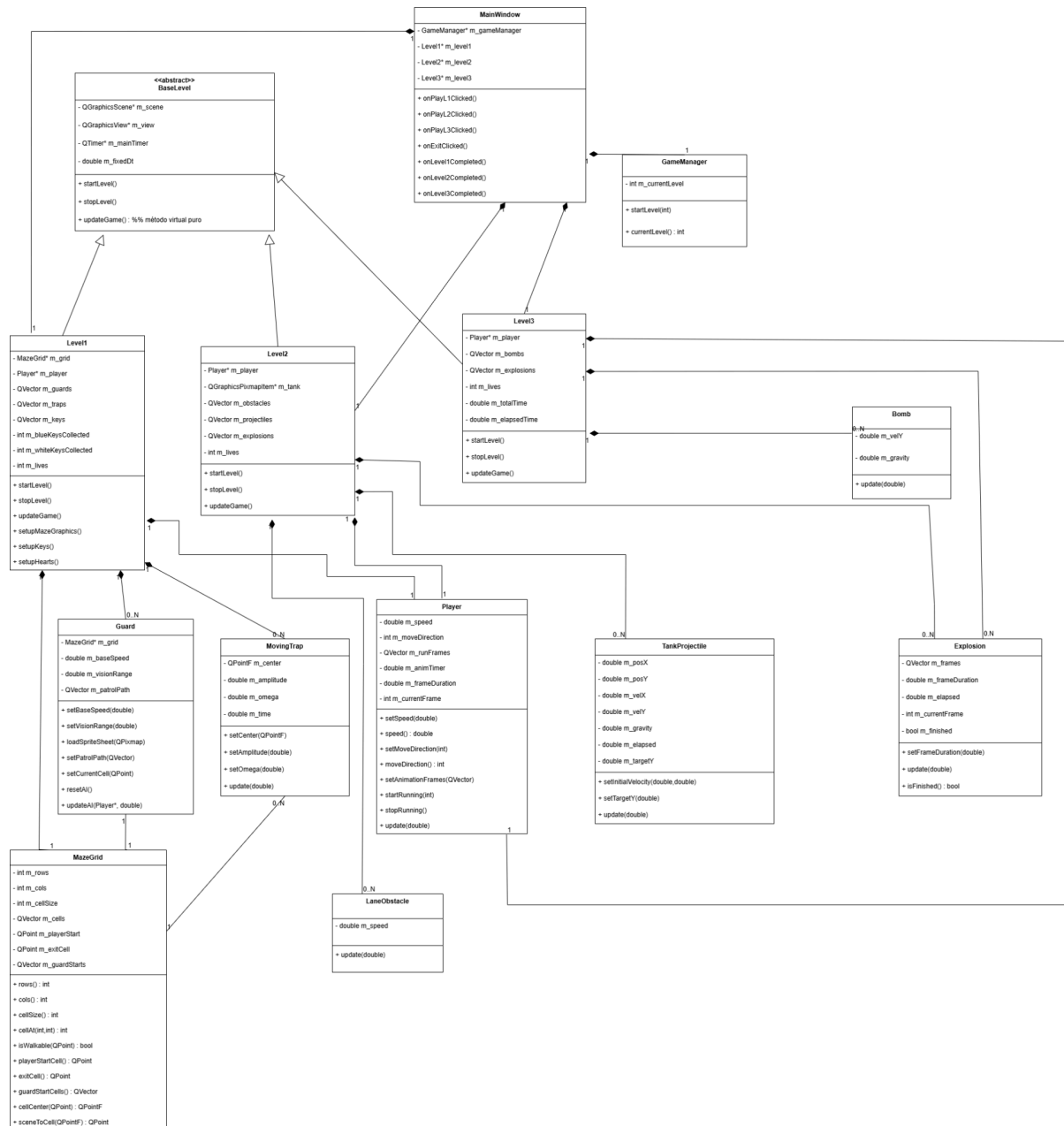
5. Metodología y planificación

El proyecto se desarrolló siguiendo una **metodología incremental** alineada con los tres momentos definidos en el enunciado:

1. **Momento I – Contextualización:**
Definición de la historia, sinopsis de cada nivel, físicas propuestas y retos.
2. **Momento II – Diseño y análisis:**
Descripción detallada de vistas, sprites, agente inteligente y un diagrama de clases de la capa lógica.
3. **Momento III – Implementación:**
Programación en Qt, integración de GUI, físicas, IA y audio; pruebas y ajustes de dificultad.

Dentro de cada momento se trabajó en ciclos pequeños: diseñar → programar → probar → ajustar (sobre todo posiciones de sprites, velocidades y colisiones).

6. Diseño y arquitectura del sistema



6.1 Componentes principales

A alto nivel el proyecto se organiza en:

- **Capa de GUI**

- **MainWindow:** controla el menú principal, los botones de selección de nivel, las pantallas de victoria/derrota y el QStackedWidget.
- Escenas QGraphicsScene y vistas QGraphicsView dentro de cada nivel.

- **Capa de lógica de juego**

- `BaseLevel`: clase base con el `QGraphicsScene`, `QGraphicsView`, temporizador principal y la función virtual `updateGame()`.
- `Level1`, `Level2`, `Level3`: heredan de `BaseLevel` e implementan la lógica específica de cada nivel (movimientos, físicas, colisiones, HUD).
- `Player`: representa al prisionero, maneja animaciones de sprites, velocidad y dirección.
- `Guard`: guardia del nivel 1 con estado “patrulla/persecución”, campo de visión y lógica de movimiento.
- `MazeGrid`: matriz de celdas del laberinto; traduce entre coordenadas de escena y celdas; decide si una celda es caminable o es pared.
- `MovingTrap`: trampa oscilatoria/circular del nivel 1.
- `LaneObstacle`: obstáculo fijo de carretera del nivel 2.
- `TankProjectile`: proyectil parabólico del tanque del nivel 2.
- `Bomb`: bomba que cae en el nivel 3.
- `Explosion`: animación de explosión reutilizada en niveles 2 y 3.
- `GameManager`: coordina el flujo entre niveles desde el menú (inicio, reinicios, etc.).

El diagrama propuesto en el Momento II mostraba una jerarquía similar basada en `Nivel`, `Entidad`, `Guardia`, `Jugador`, `Tanque` y `Proyectil`, de manera que la implementación final trasladó esa idea al modelo de clases específico de Qt, usando `QGraphicsItem` como base visual.

6.2 Dependencias externas

- **Qt 6 (MinGW 64 bits):**
 - `QtWidgets` → `QMainWindow`, `QGraphicsView`, `QGraphicsScene`, botones, textos.
 - `QtGui` → `QPixmap`, `QTransform`, manejo de sprites.
 - `QtCore` → `QTimer`, `QRandomGenerator`, contenedores, señales y slots.

- QtMultimedia → QMediaPlayer, QAudioOutput para música y efectos.
-

7. Desarrollo e implementación

Nivel 1 – Laberinto con guardias

- El laberinto se representa con MazeGrid, una matriz de celdas con tipos: pared, piso, salida, etc. Cada celda se dibuja como un QGraphicsPixmapItem usando texturas de muro, suelo y salida.
- El jugador es un Player con sprites distintos para moverse hacia arriba, abajo, izquierda y derecha. Se anima cambiando de frame cada cierto tiempo (m_playerFrameDuration).
- El movimiento se calcula en coordenadas de escena, pero cada paso se valida contra MazeGrid::isWalkable() para evitar atravesar paredes.
- Los guardias:
 - Tienen una **ruta de patrulla** (lista de celdas). Se mueven de celda a celda a velocidad constante.
 - Tienen un **campo de visión** definido por distancia máxima y una verificación de línea recta sin paredes entre el guardia y el jugador.
 - Cuando detectan al jugador, cambian a modo persecución y avanzan en dirección a la celda del jugador respetando las colisiones.
 - Si el jugador sale de su campo de visión durante cierto tiempo, el guardia “olvida” y vuelve a su ruta.
- Las trampas (MovingTrap) se mueven respecto a una posición central usando una función circular. La colisión con el jugador **resta una vida** en lugar de matar instantáneamente.
- Hay un sistema de **llaves**: al colisionar con las llaves azules/blancas se marcan como recogidas y se ocultan. La salida solo permite completar el nivel si se han recogido todas.
- Las vidas se muestran como íconos de corazón en la esquina superior derecha.

Nivel 2 – Carretera y tanque

- El fondo es una imagen ancha que se desplaza horizontalmente creando efecto de scroll. Cuando una parte sale de pantalla se vuelve a colocar, simulando desplazamiento infinito.
- El jugador corre continuamente y solo puede cambiar de carril (tres posiciones discretas en Y). Esto simula el movimiento rectilíneo sin aceleración.
- Los obstáculos de alambre se colocan en celdas específicas de los carriles.
- El tanque genera proyectiles (TankProjectile) periódicamente:
 - Se elige **aleatoriamente uno de los tres carriles** como objetivo.
 - Se calcula una trayectoria parabólica: el proyectil sale desde la posición del tanque con una velocidad inicial configurada, y su posición se actualiza según las ecuaciones de MRUA ($x(t)$, $y(t)$).
 - Se muestra una “mira” en el suelo indicando el punto de impacto; esta se destruye al terminar la caída o al colisionar con el jugador.
- Las colisiones con obstáculos o proyectiles restan vidas. Si el jugador aún tiene vidas, se reinicia su posición; si llega a cero, se emite la señal de fallo del nivel.

Nivel 3 – Trinchera con bombas

- Se usa un fondo estático tipo campo de batalla y una carretera definida con límites X.
- El jugador se mueve solamente lateralmente, con animación de carrera continua.
- Las bombas (Bomb) se crean en posiciones X aleatorias de la parte superior de la escena y caen con aceleración constante (caída libre).
- La frecuencia de aparición se incrementa con el tiempo reduciendo el intervalo del QTimer.
- Al llegar al suelo, cada bomba genera una Explosion animada usando un spritesheet; si impacta al jugador resta una vida.
- HUD:
 - Barra de tiempo naranja que se va vaciando.
 - Contador de segundos restante.

- Vidas con corazones, como en el nivel 1.

Audio

- Música de fondo en el nivel 1 (game-over-252897.mp3) y efectos de sonido al ser atrapado por un guardia (dramatic-death-collapse-352720.mp3).
 - Efecto de explosión (medium-explosion-40472.mp3) usado en la caída de bombas y proyectiles en niveles 2 y 3.
 - Implementado con QMediaPlayer + QAudioOutput.
-

8. Procedimientos de prueba

- **Pruebas manuales:**
 - Recorridos completos de cada nivel para verificar:
 - Colisiones con paredes, guardias, trampas, obstáculos, proyectiles y bombas.
 - Funcionamiento de llaves y condición de salida en el laberinto.
 - Gasto de vidas y reinicio de posiciones.
 - Temporizador y condiciones de victoria/derrota.
 - **Ajuste de parámetros:**
 - Velocidades de jugador y enemigos.
 - Rango de visión de guardias y tamaño de campo de visión.
 - Intervalos de aparición de proyectiles y bombas.
 - **Pruebas de regresión informales:**
 - Después de cada cambio grande (por ejemplo, nueva física o nueva colisión) se volvieron a probar los niveles para asegurarse de que no se rompiera el flujo del juego.
-

9. Guía de uso

1. Ejecución

- Ejecutar desde Qt Creator o lanzar el .exe generado en la carpeta que se montó a git hub llamada ejecutable en la que se encontrara un .rar que debe descomprimirse y ejecutar el .exe

2. Controles básicos

- Al momento de abrir seleccione el nivel que desee
- Nivel 1: WASD o flechas para moverse.
- Nivel 2: teclas para subir y bajar de carril W para subir y S para bajar
- Nivel 3: A y D o flechas izquierda/derecha para moverse.
- Escapar del juego mediante el botón *Exit* del menú principal o cerrando la ventana.

10. Resultados y discusión

Funcionamiento logrado

- Los tres niveles son totalmente jugables y pueden accederse desde el menú principal.
- Se implementaron las físicas requeridas: rectilínea, parabólica y caída libre, además de movimiento circular de trampas.
- El agente inteligente (guardias):
 - Percibe al jugador dentro de un campo de visión.
 - Cambia de patrulla a persecución y de nuevo a patrulla al perderlo de vista, cumpliendo las fases percepción–razonamiento–acción–“aprendizaje” básico descritas en el Momento II.
- Se añadió HUD de vidas, llaves, tiempo y pantallas visuales de éxito y game over.

Limitaciones

- La IA no implementa un algoritmo de búsqueda de rutas completo (A*, Dijkstra); el movimiento de persecución es heurístico.
- No hay sistema de guardado de progreso ni configuración avanzada.
- El juego está pensado principalmente para resolución fija; no se ha probado en múltiples tamaños de ventana.

Mejoras futuras

- Añadir niveles de dificultad seleccionables (más guardias, más velocidad, menos vidas).
- Incluir menú de opciones (volumen, controles, resolución) ya que hasta el momento solo se cuenta con un menu simple que permite acceder a los niveles y jugar.
- Implementar un algoritmo de pathfinding más sofisticado para los guardias.

11. Conclusiones

- El proyecto permitió aplicar de forma integrada conceptos de **POO**, **memoria dinámica**, diseño modular y uso de **Qt** para GUI y multimedia.
- Se logró crear un videojuego coherente, con tres niveles diferentes tanto en vista como en mecánicas, cumpliendo el requisito de diversidad de físicas y de incorporar un agente inteligente.
- El trabajo iterativo en los tres momentos ayudó a madurar la idea inicial y a refinarla durante la implementación.
- El desarrollo evidenció la importancia de:
 - Diseñar bien las clases antes de programar.
 - Mantener un ciclo constante de pruebas y ajustes.
 - Gestionar adecuadamente los recursos gráficos y sonoros mediante archivos de recursos.

12. Cambios entre momentos I-II y versión final

Al principio del desarrollo del momento 1 se quiso implementar una versión del juego muy similar a la entregada pero con algunas diferencias puntuales en las físicas y la detección, ya que se tenía pensado implementar un movimiento característico para los guardias además de una sistema de detección sofisticado como djisktra o bfs, pero debido a la complejidad de las variables implicadas como el continuo movimiento y detección dificultó el proceso, para los niveles posteriores su solución si fue fiel a lo planteado inicialmente puesto que, los únicos cambios notable fueron los escenarios, sprites y la implementación de barras de tiempo y demás componentes gráficos.

- **Nivel 1**

- En el diseño original se hablaba de que la trampa mataba al instante; en la versión final se decidió que **solo quita una vida**, igual que los guardias, para hacer la dificultad más justa.
- Se añadieron **llaves azules y blancas** como requisito adicional para abrir la salida.
- Se refinó el campo de visión de los guardias para que considere paredes (no ven a través de muros), algo que en el texto inicial estaba descrito solo de forma general.

- **Nivel 2**

- En el Momento I la idea general era “correr y esquivar proyectiles”; en la versión final se concretó el uso de **tres carriles discretos** con posiciones Y específicas en la carretera.
- Los proyectiles inicialmente se pensaron como disparos directos; en la implementación final se hizo un **disparo parabólico hacia carriles aleatorios**, con una mira visual y ajuste fino de parámetros para que realmente puedan impactar al jugador.
- Se añadió un **fondo con scroll** que no estaba especificado en el diseño original.

- **Nivel 3**

- La descripción original ya mencionaba bombas en caída libre y un tiempo límite; en la versión final se añadieron:
 - Barra de tiempo visual.
 - Animación de explosión.

- Sistema de corazones reutilizado del nivel 1.

- Se ajustó la velocidad y frecuencia de bombas para lograr una dificultad progresiva más cómoda.

- **Interfaz y flujo**

- En los primeros momentos el flujo sugerido era lineal (1→2→3). En la versión final el **menú principal permite elegir directamente cada nivel** y muestra imágenes de “nivel completado” y “game over” específicas.
- Se incorporó audio de fondo y efectos que en los momentos anteriores solo estaban planeados de forma general.